Επικοινωνία μεταξύ Διεργασιών και Σύνδρομες Διεργασίες

Interprocess Communication and Concurrent Processes

Περίληψη

- Σύνδρομος Προγραμματισμός
  - Συνθήκη συναγωνισμού
  - Συγχρονισμός διεργασιών
- Κρίσιμες περιοχές και αμοιβαίος αποκλεισμός
- Το Πρόβλημα Παραγωγών και Καταναλωτών
- Λύσεις με
  - Σηματοφόρους
  - Μηνύματα

Σύνδρομος Προγραμματισμός

- Σύνδρομος Προγραμματισμός (concurrent programming): ταυτόχρονη εκτέλεση δύο ή περισσοτέρων διεργασιών σε ένα σύστημα.
  - Πολυπρογραμματισμός (συνεχής εναλλαγή διεργασιών που εκτελούνται από τον επεξεργαστή)
  - Συστήματα πολλαπλών επεξεργαστών
- Προβλήματα
  - Επικοινωνία μεταξύ διεργασιών
  - Καταμερισμός πόρων μεταξύ διεργασιών
    - Ανταγωνιζόμενες διεργασίες
    - Συνεργαζόμενες διεργασίες
  - Συγχρονισμός των δραστηριοτήτων μεταξύ διεργασιών
- Τα προβλήματα εμφανίζονται στους τομείς
  - Πολλαπλών εφαρμογών
  - Δομή εφαρμογών (εφαρμογές δομημένες σαν πολλαπλές διεργασίες ή υποδιεργασίες)
  - Δομή του ΛΣ (εφαρμογή του ΛΣ σαν πολλαπλές διεργασίες)

Σύνδρομες Διεργασίες

- Το ΛΣ (σε συνεργασία με το υλικό (hardware) του συστήματος) είναι υπεύθυνο για την επίλυση των πιο κάτω προβλημάτων
  - Διαχείριση των εκτελούμενων διεργασιών και κατανομή πόρων
    - Μια διεργασία μπορεί να μην γνωρίζει την ύπαρξη της άλλης και έτσι να **ανταγωνίζονται** για τον καταμερισμό των πόρων (competition)
    - Μια διεργασία μπορεί να γνωρίζει έμμεσα την ύπαρξη της άλλης (π.χ., ύπαρξη κοινής μνήμης) οπόταν μπορεί να **συνεργάζονται** (collaborate)
    - Μια διεργασία μπορεί να γνωρίζει άμεσα την ύπαρξη της άλλης (π.χ., έχουν σχεδιαστεί μαζί) οπόταν μπορεί να **συνεργάζονται** (collaborate).
  - Προστασία του χώρου της μιας διεργασίες από παρεμβολές από άλλες διεργασίες.
    - Παροχή τρόπων επικοινωνίας και συγχρονισμού
  - **Ανεξαρτησία αποτελεσμάτων από τη σχετική ταχύτητα εκτέλεσης**

Απλό Παράδειγμα

- Υποθέστε ότι δύο διεργασίες Δ1 και Δ2 χρησιμοποιούν τον ακόλουθο κώδικα για να «ξανατυπώσουν» κάθε χαρακτήρα εισόδου στην έξοδο (οθόνη).

```
void echo()
{
        chIn= getChar();
        chOut= chIn;
        putChar(chOut);
}
```

- Για εξυπηρέτηση χώρου υπάρχει μόνο ένα αντίγραφο στη μνήμη το οποίο τρέχουν όλες οι διεργασίες.
- Ποιο το αναμενόμενο αποτέλεσμα του κώδικα;

Απλό Παράδειγμα

- Υποθέστε ότι οι είσοδος της Δ1 είναι x και την Δ2 y.
- Βρέστε ένα πιθανό ίχνος

```
void echo()
{
        chIn= getChar();
        chOut= chIn;
        putChar(chOut);
}
```

- Διεργασία 1
  - `chIn= getChar(); //=x`
  -
  -
  -
  - `chOut= chIn;  //=y`
  - `putChar(chOut);`
- Διεργασία 2
  -
  - `chIn= getChar(); //=y`
  - `chOut= chIn;  //=y`
  - `putChar(chOut);`
  -

- Πρόβλημα: Το y τυπώθηκε δύο φορές ενώ το x καμιά.

Συνεργασία Διεργασιών: Παράδειγμα 1

- Υποθέστε ότι οι Δ1 και Δ2 έχουν κοινές μεταβλητές `a=1` και `b=2`.
  - Δ1: `a= a+b;`
  - Δ2: `b= a+b;`
- Βρέστε τις τελικές τιμές των μεταβλητών `a` και `b`.

- Διεργασία 1
  - `a= 1+2 =3`
  -
- Διεργασία 2
  -
  - `b= 3+2= 5`

- Διεργασία 1
  -
  - `a= 1+3= 4`
- Διεργασία 2
  - `b= 1+2 =3`
  -

- Πρόβλημα: Η σειρά με την οποία εκτελούνται οι εντολές επηρεάζει το τελικό αποτέλεσμα (race conditions).

Συνεργασία Διεργασιών: Παράδειγμα 1

- Υποθέστε ότι οι Δ1 και Δ2 έχουν κοινές μεταβλητές `a=1` και `b=1`.
  - Διεργασία 1
    - `a= a+1;`
    - `b= b+1;`
  - Διεργασία 2
    - `b= 2*b;`
    - `a= 2*a;`
- Βρέστε τις τελικές τιμές των μεταβλητών `a` και `b`.

Δ1	Δ2
`a= 1+1=2`	
`b= 1+1=2`	
	`b=2*2=4`
	`a=2*2=4`

Δ1	Δ2
`a= 1+1=2`	
	`b=2*1=2`
`b= 2+1=3`	
	`a=2*2=4`

- Πρόβλημα: Η σειρά με την οποία εκτελούνται οι εντολές επηρεάζει το τελικό αποτέλεσμα (race conditions).

Συνθήκη Συναγωνισμού (Race Conditions)

- Η σειρά με την οποία θα εκτελεστούν δύο ή περισσότερες εντολές δεν είναι καθορισμένη.
  - Μπορεί να αλλάξει κάτω από συνθήκες μη ελεγχόμενες από τον προγραμματιστή
- Προγραμματιστικά λάθη αυτής της μορφής είναι πολύ δύσκολα να ανιχνευτούν και να διορθωθούν αφού η συμπεριφορά του προγράμματος μπορεί να αλλάξει από εκτέλεση σε εκτέλεση.

Κρίσιμες Περιοχές και Αμοιβαίος Αποκλεισμός

- **Κρίσιμες περιοχές** (critical region): μέρος του προγράμματος στο οποίο επιχειρείται πρόσβαση σε κοινό χώρο, κοινά αρχεία κλπ.
- **Αμοιβαίος Αποκλεισμός** (mutual exclusion): Μηχανισμός ο οποίος δεν επιτρέπει την ταυτόχρονη πρόσβαση σε κρίσιμες περιοχές από δύο ή περισσότερες διεργασίες.
- Για να επιτύχουμε αποδοτικό συντονισμό μεταξύ παράλληλων διεργασιών οι ακόλουθες συνθήκες πρέπει να ισχύουν
  - Δεν πρέπει να βρίσκονται ταυτόχρονα σε κρίσιμη περιοχή περισσότερες από μία διεργασίες.
  - Το αποτέλεσμα πρέπει να είναι ανεξάρτητο από τη ταχύτητα των επεξεργαστών ή τον καταμερισμό του χρόνου του επεξεργαστή.
  - Διεργασία που τρέχει εκτός κρίσιμης περιοχής δεν μπορεί να αποκλείσει άλλη διεργασία
  - Καμιά διεργασία δεν πρέπει να μπορεί να αποκλειστεί για πάντα.

Προστασία με Αμοιβαίο Αποκλεισμό

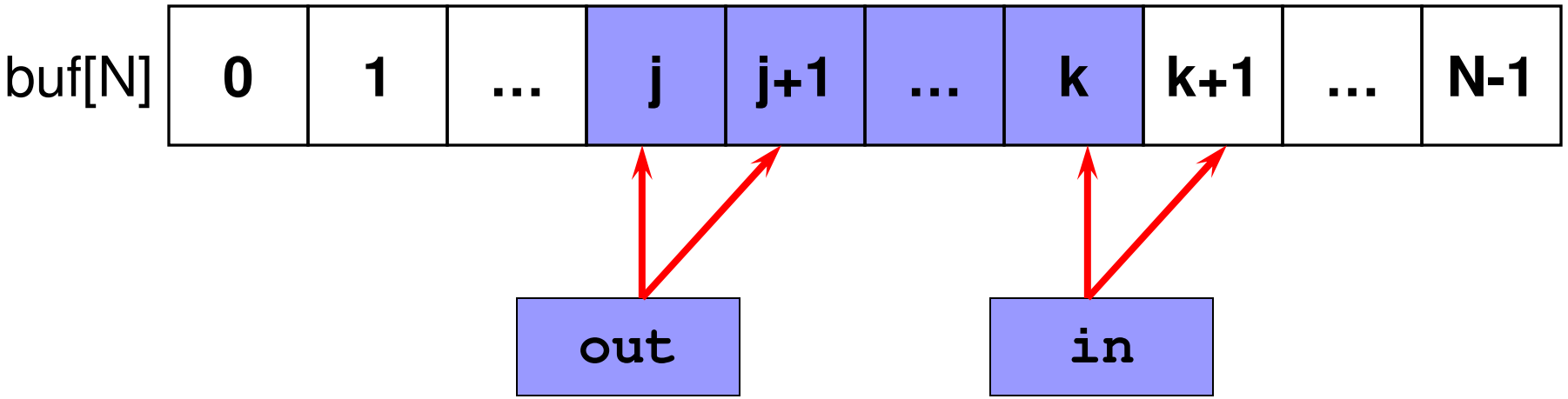
- Υποθέστε ότι οι Δ1 και Δ2 έχουν κοινές μεταβλητές `a=1` και `b=1`.
  - Διεργασία 1
    - `enterCritical();`
    - `a= a+1;`
    - `b= b+1;`
    - `exitCritical();`
  - Διεργασία 2
    - `enterCritical;`
    - `b= 2*b;`
    - `a= 2*a;`
    - `exitCritical();`

- Η εντολή `enterCritical()` δεν επιτρέπει σε άλλη διεργασία πρόσβαση στον προστατευμένο χώρο.
- Πρόσβαση επιτρέπεται μόνο μετά την εκτέλεση της `exitCritical()`.
- Με αυτό τον τρόπο εγγυούμαστε τη σωστή εκτέλεση

Δ1	Δ2
`a= 1+1=2`	
`b= 1+1=2`	
	`b=2*2=4`
	`a=2*2=4`

Πρόβλημα Παραγωγών και Καταναλωτών

- Υποθέστε ότι
  - n διεργασίες παράγουν (produce) κάποια δεδομένα τα οποία αποθηκεύουν σε μια κυκλική ενδιάμεση μνήμη (ουρά).
  - m διεργασίες καταναλώνουν τα δεδομένα (consume)




```
array buf[N];
int in, out;    /*global variables*/
```

Πρόβλημα Παραγωγών και Καταναλωτών

```
array buf[N];
int in, out;   /*global variables*/
```

```
void producer()
{
   while(true)
   {
      item= produce();
      buf[in]= item;
      in= (in+1)%N;
   }
}
```

```
void consumer()
{
   while(true)
   {
      item= buf[out];
      consume(item);
      out= (out+1)%N;
   }
}
```

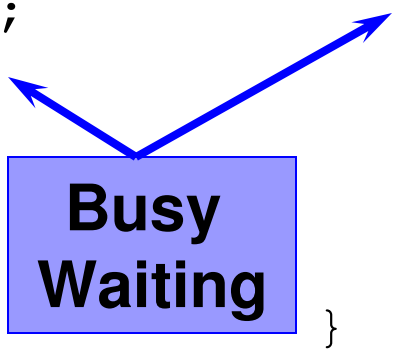
- Προβλήματα:
  - Τι θα συμβεί αν η διαδικασία `consumer()` τρέξει όταν το buffer είναι άδειο;
  - Τι θα συμβεί αν η διαδικασία `producer()` τρέξει όταν το buffer είναι γεμάτο;

Πρόβλημα Παραγωγών και Καταναλωτών

```
array buf[N];
int nBuf=0;          /*number of items in buffer*/
int in, out;         /*global variables*/

void producer()
{
   while(true)
   {
      item= produce();
      while(nBuf==N);
      buf[in]= item;
      in= (in+1)%N;
      nBuf++;
   }
}

void consumer()
{
   while(true)
   {
      while(nBuf==0);
      item= buf[out];
      consume(item);
      out= (out+1)%N;
      nBuf--;
   }
}
```



- Προβλήματα: συγχρονισμός των διεργασιών

Πρόβλημα Συγχρονισμού

```
void producer()
{
    while(true)
    {
        item= produce();
        while(nBuf==N);
        buf[in]= item;
        in= (in+1)%N;
        nBuf++;
    }
}
```

- Υποθέστε ότι
  - out= 5
  - in= 7
  - nBuf= 2

Producer 1	Producer 2
Buf[7]= item1;	
	Buf[7]= item2;
	in=7+1= 8
	nBuf= 3
in=7+1= 8	
nBuf= 3	

Υποθέτουμε ότι οι τιμές του `in` και `nBuf` αποθηκεύονται σε κάποια τοπική μεταβλητή (καταχωρητή).

Αμοιβαίος Αποκλεισμός (Mutual Exclusion)

- Κρίσιμη Περιοχή (critical region)
- Σε κάθε δεδομένη στιγμή, μόνο μια διεργασία μπορεί να βρίσκεται μέσα στην κρίσιμή περιοχή

```
void producer()
{
    while(true)
    {
        item= produce();
        while(nBuf==N);
        buf[in]= item;
        in= (in+1)%N;
        nBuf++;
    }
}
```

```
void consumer()
{
    while(true)
    {
        while(nBuf==0);
        item= buf[out];
        consume(item);
        out= (out+1)%N;
        nBuf--;
    }
}
```

Κρίσιμη Περιοχή

Εφαρμογή Αμοιβαίου Αποκλεισμού

- Υποθέστε πως υπάρχουν μόνο δύο διεργασίες.

```
int turn=0;   /*global variable*/
void process0()
{
    …
    while(turn!=0);
    /*criticalRegion*/
    turn= 1;
    …
}
```

```
void process1()
{
    …
    while(turn!=1);
    /*criticalRegion*/
    turn= 0;
    …
}
```

- Πετυχαίνεται ο αμοιβαίος αποκλεισμός; **Ναι!**
- Λύθηκε το πρόβλημα; **Όχι!**
  - Τι θα συμβεί εάν οι δύο διεργασίες εκτελούν την κρίσιμη περιοχή με διαφορετική συχνότητα;
  - Διεργασία που τρέχει εκτός κρίσιμης περιοχής δεν αφήνει άλλη διεργασία να μπει στη κρίσιμη περιοχή.

Εφαρμογή Αμοιβαίου Αποκλεισμού

```
boolean interested[2];   /*global variable*/
void process0()              void process1()
{                            {
    …                            …
    while(interested[1]);        while(interested[0]);
    interested[0]= true;         interested[1]= true;
    /*criticalRegion*/           /*criticalRegion*/
    interested[0]= false;        interested[1]= false;
    …                            …
}                            }
```

- Πετυχαίνεται ο αμοιβαίος αποκλεισμός; **Όχι!**
  - Η διεργασία 0 εκτελείται μέχρι το while loop, βρίσκει το interested[1]=true και αμέσως διακόπτεται.
  - Η διεργασία 1 μπαίνει κανονικά στη κρίσιμη περιοχή αφού interested[0]=false.
  - Στη συνέχεια μπαίνει και η διεργασία 0 στη κρίσιμη περιοχή.

Εφαρμογή Αμοιβαίου Αποκλεισμού Λύση Peterson

```
int turn= 0;
boolean interested[2];   /*global variable*/

void process0()
{
    …
    interested[0]= true;
    turn= 0;
    while(turn==0 &&
          interested[1]);
    /*criticalRegion*/
    interested[0]= false;
    …
}
```

```
void process1()
{
    …
    interested[1]= true;
    turn= 1;
    while(turn==1 &&
          interested[0]);
    /*criticalRegion*/
    interested[1]= false;
    …
}
```

- Πετυχαίνεται ο αμοιβαίος αποκλεισμός με χρήση λογισμικού.
- Μειονέκτημα: χρήση του busy-waiting

Σηματοφόροι (Semaphores)

```
struct semaphore
{
    int count;
    queueType queue;
}
```

- Οι Σηματοφόροι είναι δομές υλοποιούνται από το πρόγραμμα μεταγλωττισμού (compiler) και οι οποίες υποστηρίζονται από το ΛΣ.

```
void down(semaphore s)
{
    s.count--;
    if(s.count < 0)
    {
        addProcess(s.queue);
        sleep();
    }
}
```

```
void up(semaphore s)
{
    s.count++;
    if(s.count <= 0)
    {
        p= getProcess(s.queue);
        wakeup(p);
    }
}
```

- Δυαδικοί σηματοφόροι (binary semaphores – mutex)

Σηματοφόροι (Semaphores)

- Οι Σηματοφόροι αρχικοποιούνται σε κάποια μη-αρνητική τιμή
- Η ρουτίνα down() (ή wait()) μειώνει την τιμή κατά 1.
  - Εάν η τιμή δεν είναι αρνητική τότε η διεργασία συνεχίζει κανονικά
  - Εάν η τιμή γίνει αρνητική τότε σταματά η εκτέλεση της διεργασίας
- Η ρουτίνα up() (ή signal()) αυξάνει την τιμή κατά 1.
  - Εάν η τιμή γίνει μηδέν ή παραμείνει αρνητική τότε μια διεργασία επανανεργοποιείται και γίνεται έτοιμη για εκτέλεση.
- Οι Δυαδικοί Σηματοφόροι αρχικοποιούνται στη τιμή 0 ή 1
- Η ρουτίνα down() (ή wait()) ελέγχει την τιμή.
  - Εάν η τιμή είναι 1, τότε την μετατρέπει σε 0 η διεργασία συνεχίζει κανονικά
  - Εάν η τιμή είναι 0 τότε σταματά η εκτέλεση της διεργασίας
- Η ρουτίνα up() (ή signal())
  - Ελέγχει εάν κάποια διεργασία είναι απενεργοποιημένη και την επανανεργοποιεί
  - Εάν δεν υπάρχει απενεργοποιημένη διεργασία μετατρέπει την τιμή σε 1.

Αμοιβαίος Αποκλεισμός με Σηματοφόρους

```
semaphore lock= 1;          /*global variable*/
int N;                      /*Number of processes*/

void process(int i)
{
    while(true)
    {
        …
        down(&lock);
        /*criticalRegion*/
        up(&lock);
        …
    }
}
```

- Πετυχαίνεται ο αμοιβαίος αποκλεισμός με την χρήση σηματοφόρων.

Το Πρόβλημα Παραγωγού-Καταναλωτή με Σηματοφόρους

```
semaphore lock= 1;
semaphore empty= N;      /*buffer size*/
semaphore full= 0;
void producer()
{
    while(true)
    {
        item= produce();
        down(&empty);
        down(&lock);
        buf[in]= item;
        in= (in+1)%N;
        nBuf++;
        up(&lock);
        up(&full);
    }
}
```

```
void consumer()
{
    while(true)
    {
        down(&full);
        down(&lock);
        item= buf[out];
        out= (out+1)%N;
        nBuf--;
        up(&lock);
        up(&empty);
        consume(item);
    }
}
```

Κρίσιμη Περιοχή

Ανταλλαγή Μηνυμάτων

- Τι θα συμβεί σε περίπτωση που έχουμε ένα κατανεμημένο σύστημα με πολλούς επεξεργαστές οι οποίοι επικοινωνούν μέσω δικτύου;
- ΛΣ υποστηρίζουν την ανταλλαγή μηνυμάτων μέσω συστημικών κλήσεων.
  - `send(destination, &message);`
  - `receive(source, &message);`
- Πολλαπλά προβλήματα
  - Τι θα συμβεί εάν το μήνυμα χαθεί;
  - Τι θα συμβεί εάν χαθεί η επιβεβαίωση λήψης (Acknowledgement΄);
  - Πιστοποίηση ταυτότητας (authentication).
  - Τα προβλήματα αυτά είναι το θέμα του μαθήματος ΗΜΠ 337 Δίκτυα Υπολογιστών

Το Πρόβλημα Παραγωγού-Καταναλωτή με Μηνύματα

```
void Producer()
{
   int item;
   message m;
   while(true)
   {
      item= produce();
      receive(consumer, &m);
      buildMessage(&m,item);
      send(consumer,&m);
   }
}
```

```
void consumer()
{
   int item;
   message m;
   for(int i=0; i<N; i++)
        send(producer, &m);

   while(true)
   {
      receive(producer, &m);
      item= getItem(&m);
      send(producer,&m);
      consume(item);
   }
}
```